ProTech
protechtraining.com

Slide 1

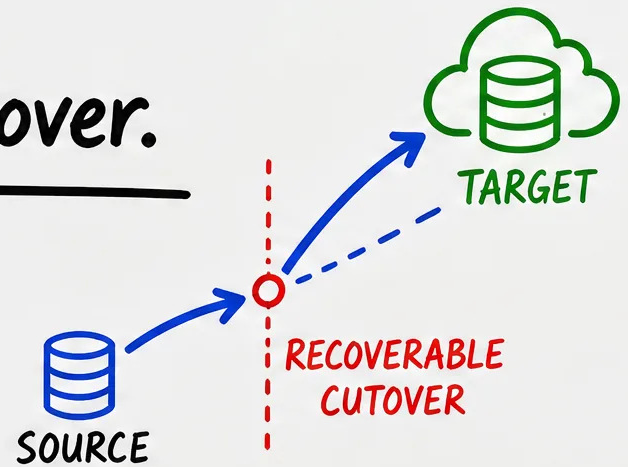
Module 20 – Migration



TESTED TARGET

Tested target. Recoverable cutover.

Migration is a controlled change.



ASSESS → PLAN → MIGRATE → VALIDATE → CUTOVER → OPTIMIZE

Two Workstreams, One Release Decision

Applications, deployment descriptors, dependencies, and tests travel on one workstream; profiles, resources, security, and topology travel on another.

Workstream 1:
Application Delivery

- Applications
- Deployment Descriptors
- Dependencies
- Tests



Workstream 2:
Runtime Configuration

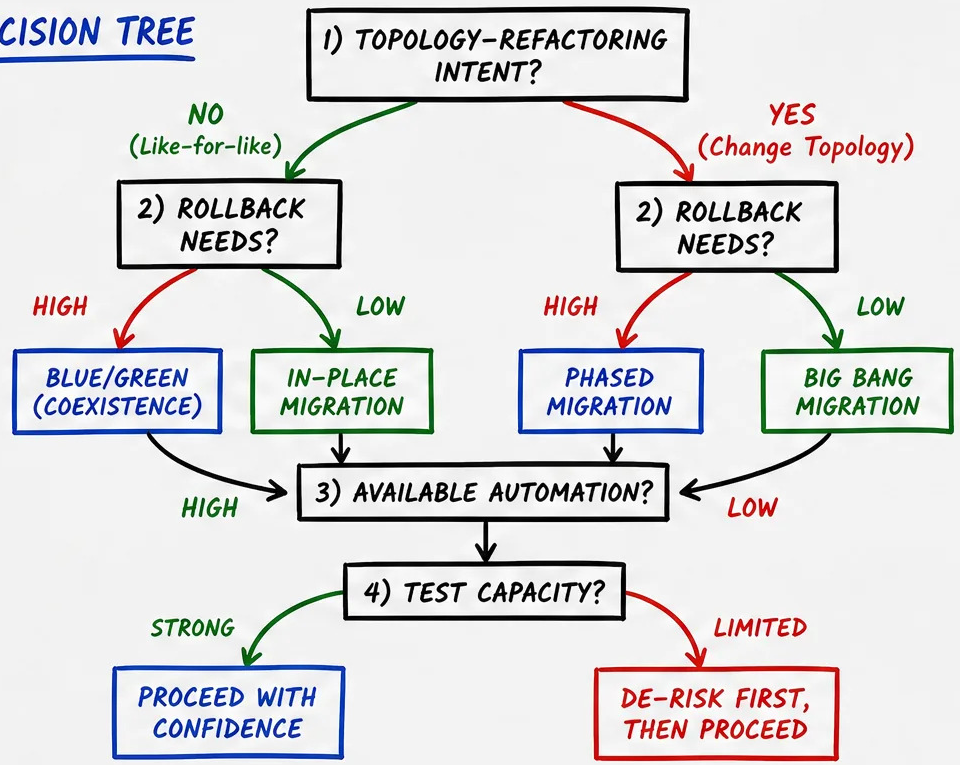
- Profiles
- Resources
- Security
- Topology

Application Code and **Runtime Configuration** take separate paths to **the same target.**

Choose the Migration Strategy Deliberately

Use a decision tree based on topology-refactoring intent, rollback needs, available automation, and test capacity; do not treat a legacy diagram as a runbook.

Decision Tree



What This Means

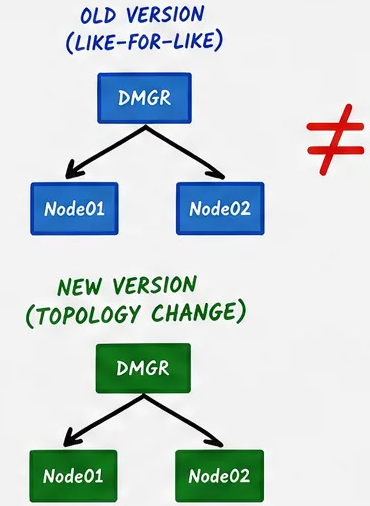
TOPOLOGY-REFACTORING INTENT
Decide if you're changing topology or keeping it the same.

ROLLBACK NEEDS
High rollback needs favor coexistence or phased approaches.

AVAILABLE AUTOMATION
More automation increases confidence and execution speed.

TEST CAPACITY
Stronger test capacity enables safer, faster migrations.

Don't Confuse Reference with Prescription



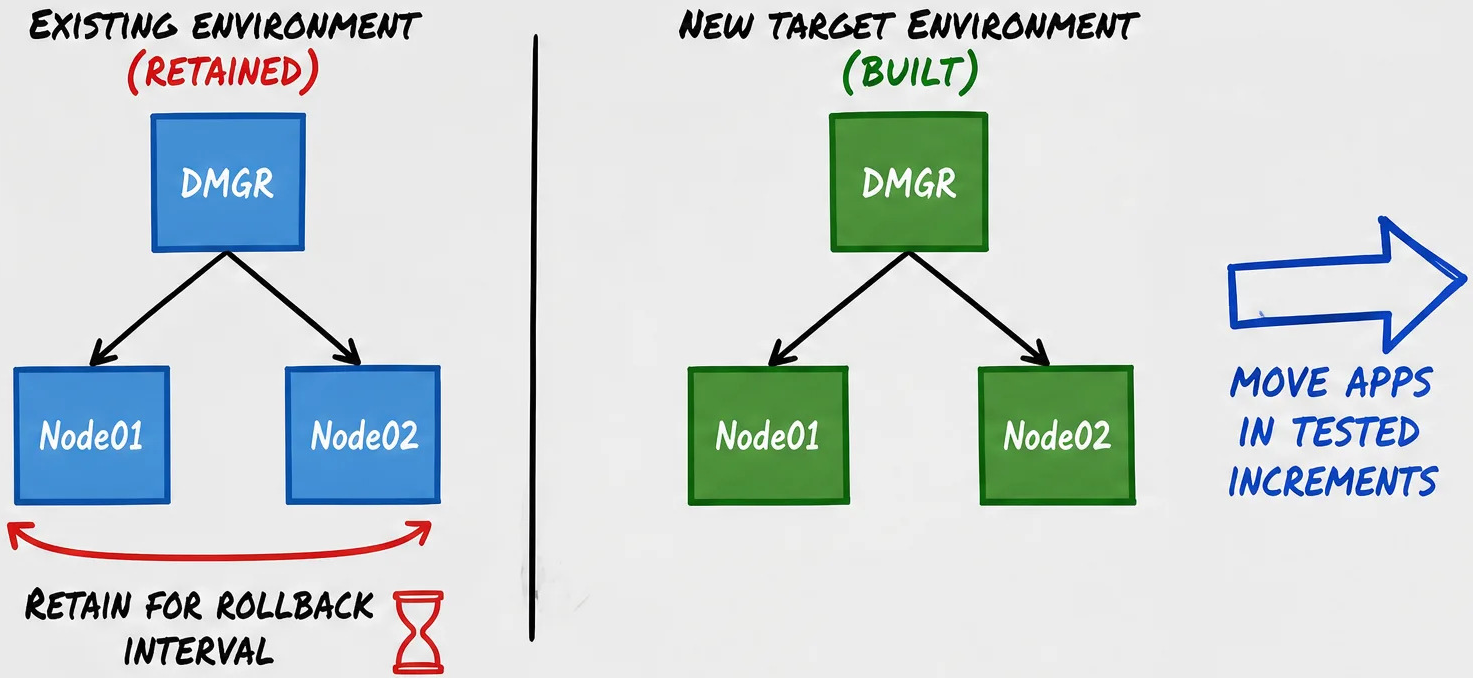
DIAGRAMS SHOW POSSIBILITIES, NOT STEP-BY-STEP RUNBOOKS.

THE RIGHT STRATEGY MINIMIZES RISK, NOT JUST EFFORT.

Side-by-Side: Build, Test, Cut Over

4

Build a new target environment, move applications in tested increments, and retain the earlier environment for a governed rollback interval.



1. Build
Build the new target environment.

2. Test
Validate and test application increments.

3. Cut Over
Move applications in tested increments.

4. Retain & Rollback
Retain the earlier environment for a governed rollback interval.

Memorable Statement:

Side-by-side migration treats the infrastructure as **immutable**.

Automated Configuration Migration: Backup Then Restore

5

Use Version 9.0 tools to save a supported source configuration and applications to a migration backup, then import that content into the installed target; use the target-version tools.

SOURCE (v9.0)



- ✓ Supported configuration
- ✓ Applications

EXPORT (v9.0 TOOLS) →

MIGRATION BACKUP



BACKUP PACKAGE (CONFIG + APPS)

IMPORT (TARGET TOOLS) →

TARGET (INSTALLED)



- ✓ Imported configuration
- ✓ Imported applications

MEMORABLE STATEMENT:
TRUST THE MIGRATION TOOLS TO TRANSLATE THE XML, BUT **VERIFY** THE RESULT.

VERIFY THE RESULT



Coexistence Needs Port and Ownership Discipline

6

Different WebSphere releases can run at the same time, but unique port assignments, distinct service ownership, and a tested traffic plan prevent bind conflicts and accidental cross-use.

Memorable Statement

TWO JVMs CANNOT BIND TO THE SAME SOCKET.

Coexistence Rules

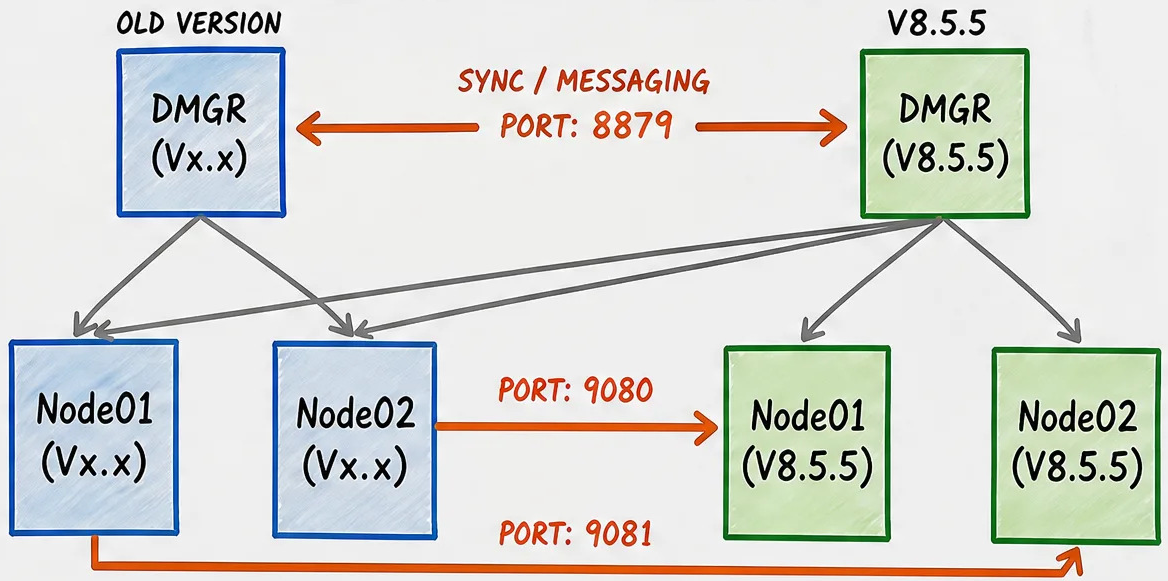
UNIQUE PORTS
No two JVMs share the same port.

OWNERSHIP
Each service and port has a clear owner.

TESTED PLAN
A validated traffic plan prevents conflicts.

DISCIPLINE
Follow standards, document, and enforce.

Coexistence Example



UNIQUE PORTS + CLEAR OWNERSHIP + TESTED PLAN = SMOOTH COEXISTENCE

Stage the Topology, Not the Risk

7

Use temporary staging only when it reduces cutover risk, with an explicit end state, compatibility review, and rollback boundary; verify source/target operating-system and architecture conditions for remote tooling.

Use staging when it reduces risk

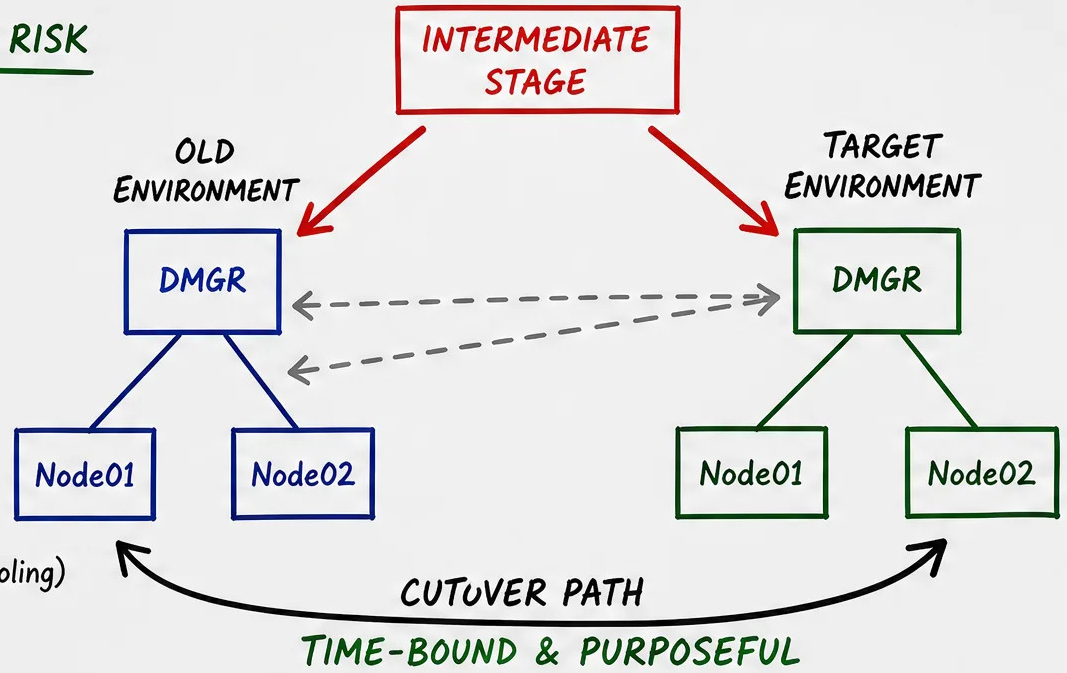
- Rehearse complex changes
- Validate automation & tooling
- Build confidence before cutover

Require explicit end state

- Define the final topology
- Time-bound staging
- No permanent detours

Protect the cutover

- Compatibility review (OS, architecture, tooling)
- Clear rollback boundary
- Document assumptions & limits



Verify compatibility

CHECK	VERIFY
Operating System	✓
Architecture (x86_64 / ARM)	✓
Remote Tooling (SSH, Agents, APIs)	✓
Version Compatibility	✓
Network Connectivity	✓

Memorable statement:

Staging environments should be **rehearsals**, not permanent complexity.

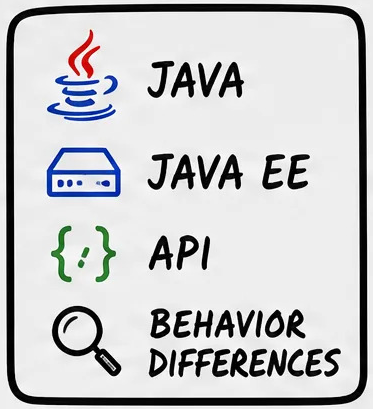
Application Readiness: Assess Before You Move

1. Scan



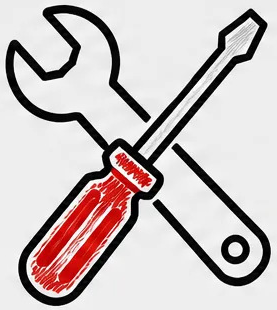
Scan source or binaries

2. Identify



Identify Java, Java EE, API, and behavior differences

3. Remediate



Remediate findings

4. Build & Test



Build and test on the target runtime

5. Target Runtime



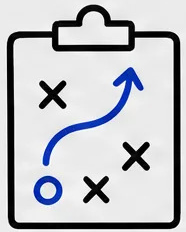
Confident. Compatible. Ready.



Do not discover application incompatibilities in production.

Lab: Rehearse a Recoverable Migration

1. Plan



- Scope
- Approach
- Runbook

2. Back up



- Data
- Configurations
- Dependencies

3. Migrate or Build



- Migrate or Build
- Configure
- Integrate

4. Deploy



- Release
- Monitor
- Stabilize

5. Validate



- Business Transactions
- Operations
- Performance

6. Cut over only on evidence



- Evidence-Based
- Go / No-Go
- Communications

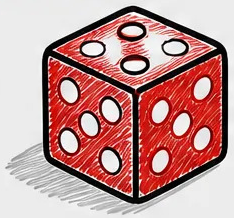
7. Exercise Rollback Criteria



- Trigger Criteria
- Rollback Plan
- Recovery Steps



- Time Targets
- Communication
- Verify Recovery



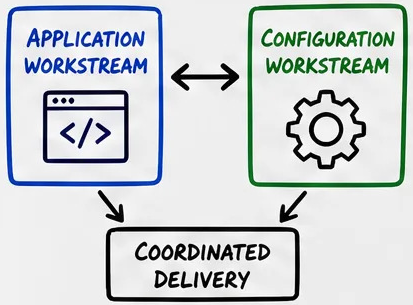
A migration without a Rollback Rehearsal is just gambling.

Migration Decision Checklist

10

A disciplined checklist for a successful, low-risk migration.

1 Application vs. Configuration Workstreams



- Separate focus, shared outcomes
- Plan, coordinate, and align

2 Migration vs. Clean-Build Strategy



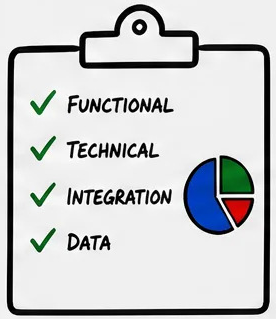
- Compare effort, risk, time, and value
- Choose the strategy that best fits your goals

3 Coexistence Controls



- Isolation & security
- Data segregation
- Integration boundaries
- Rollback readiness

4 Application Assessment



- Inventory & dependencies
- Complexity & effort
- Remediation plan
- Migration readiness

5 Rehearsal



- Dry runs build confidence
- Validate processes
- Find & fix issues early
- Refine and repeat

6 Current Fix-Pack Validation



- Latest fix-pack level
- Vendor certified
- Prerequisites met
- Reduce risk & defects



Treat every migration step as a Measurable Gateway.

